

SQL - Les bases

Interroger des donnees, simplement.



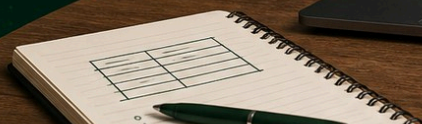
Tables



Requetes



Filtres



FORMATION SQL - LES BASES

SQL

Interroger des tables et lire des resultats,
explique simplement.

DanielCraft - Livre debutant - Pack Data/SQL

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - C'est quoi SQL ?

- [Ce que ce n'est pas](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Exemple pour sentir](#)

2. Chapitre 2 - SELECT : lire des colonnes

- [Ce que ce n'est pas](#)
- [Alias et lisibilité](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. Chapitre 3 - WHERE : filtrer les lignes

- [Combiner : AND et OR](#)
- [IN et LIKE \(léger\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. Chapitre 4 - ORDER BY et LIMIT

- [Où placer les clauses](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. Chapitre 5 - NULL : la valeur absente

- [NULL dans les calculs et filtres](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. Chapitre 6 - Agregats et GROUP BY

- [GROUP BY](#)
- [HAVING \(aperçu\)](#)
- [Petite histoire](#)
- [Erreur classique](#)

- [En vrai](#)
- [A toi](#)

7. [Chapitre 7 - INNER JOIN : croiser deux tables](#)

- [Alias de tables](#)
- [Pourquoi INNER "perd" des lignes](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

8. [Chapitre 8 - LEFT JOIN : garder la gauche](#)

- [Trouver ceux sans match](#)
- [INNER vs LEFT : choisir](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

9. [Chapitre 9 - INSERT et UPDATE](#)

- [UPDATE : le WHERE obligatoire mentalement](#)
- [Plusieurs colonnes](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

10. [Chapitre 10 - DELETE : retirer des lignes](#)

- [Le reflexe SELECT miroir](#)
- [DELETE sans WHERE](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

11. [Chapitre 11 - Cles primaires et etrangeres](#)

- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

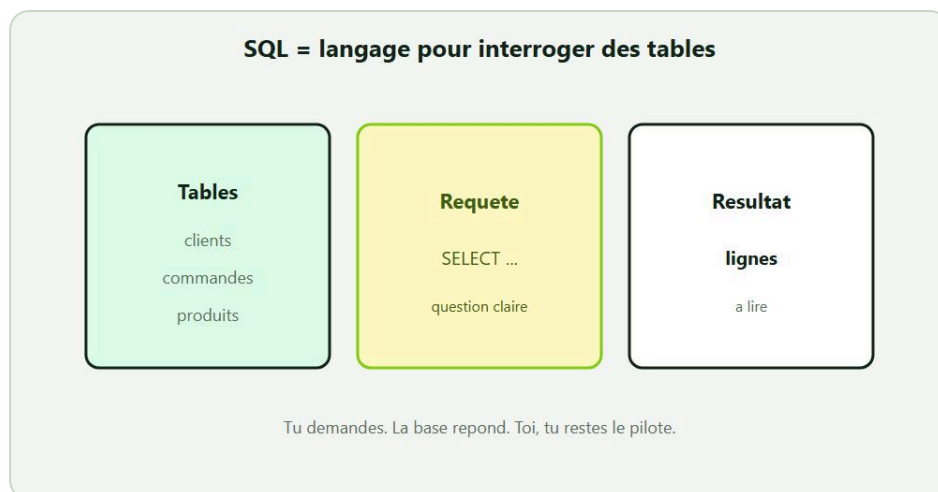
12. [Chapitre 12 - Types de colonnes](#)

- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

13. [Chapitre 13 - Mini-projet : clients et commandes](#)

- [Schema invente](#)
 - [Questions a repondre](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [A toi](#)
14. [Chapitre 14 - Retenir l'essentiel](#)
- [Carte courte](#)
 - [A toi](#)
15. [Chapitre 15 - Atelier : SELECT](#)
- [Etapas](#)
 - [A toi](#)
16. [Chapitre 16 - Atelier : filtrer avec WHERE](#)
- [Missions](#)
 - [A toi](#)
17. [Chapitre 17 - Atelier : jointure](#)
- [Missions](#)
 - [A toi](#)
18. [Chapitre 18 - Erreurs classiques](#)
- [Petite histoire](#)
 - [A toi](#)
19. [Chapitre 19 - Bonnes pratiques](#)
- [A toi](#)
20. [Quiz final](#)
- [Question 1](#)
 - [Question 2](#)
 - [Question 3](#)
 - [Question 4](#)
 - [Question 5](#)
 - [Question 6](#)
 - [Question 7](#)
 - [Question 8](#)
 - [Question 9](#)
 - [Question 10](#)
 - [Question 11](#)
 - [Question 12](#)
 - [Corrige](#)
21. [Bravo](#)
- [Suite](#)
 - [A toi](#)

Chapitre 1 - C'est quoi SQL ?



SQL : poser une question claire a des tables.

Tu as deja vu un tableau Excel, une liste de clients, un export de commandes. **SQL** (Structured Query Language) est le langage pour **poser des questions** a une base de donnees relationnelle : des tables rangees en lignes et colonnes. Tu ecris une **requete**. La base repond avec un resultat. Tu restes le pilote. Chez DanielCraft, on presente SQL comme un filtre clair - pas comme de la magie noire. Lea l'utilise pour verifier qui a commande quoi. Max a compris le jour ou il a filtre "Lyon" au lieu de scroller cent lignes. Sam dit a ses eleves : "SQL ne remplace pas ton jugement. Il execute ta question."

Le geste mental est simple. Une table `clients` ressemble a une grille : `id`, `nom`, `ville`. Une table `commandes` a `id`, `client_id`, `total`, `cree_le`. Tu demandes : "montre-moi les clients de Lyon". La base lit, filtre, renvoie. Ce livre suit le meme rythme que les autres formations DanielCraft : petit, clair, testable. On suppose que tu sais ouvrir un editeur et lire un resultat. Pas besoin d'etre DBA. Besoin de curiosite.

★ A RETENIR

SQL = langage pour interroger (et parfois modifier) des tables. Tu poses une question claire. La base repond. Toi, tu verifies.

Ce que ce n'est pas

Ce n'est pas Excel avec des formules. Ce n'est pas Python. Ce n'est pas un ORM magique qui "sait" ce que tu veux. Ce n'est pas obligatoire de connaitre toutes les bases (PostgreSQL, MySQL, SQLite) avant la premiere requete : la syntaxe de base se ressemble beaucoup. Ce n'est pas non plus "tout memoriser avant SELECT". Commence par lire. Ensuite ecrire.

Ce n'est pas non plus "big data" ni "data science" des le jour un. Lea rappelle : le vrai travail, c'est formuler une question honnete. Les mots-cles aident a ne pas se tromper de grille.

Ce que tu vas savoir faire

A la fin de ce livre, tu sauras lire avec `SELECT`, filtrer avec `WHERE`, trier et limiter, comprendre `NULL`, agreger avec `COUNT` / `SUM` / `GROUP BY`, joindre avec `INNER JOIN` et `LEFT JOIN`, inserer et mettre a jour avec prudence, supprimer sans catastrophe, comprendre cles primaires et etrangeres, et livrer un mini rapport clients/commandes. Niveau debutant solide. Pas d'administration serveur avancee. Juste du SQL clair.

Comment lire ce livre

Lis dans l'ordre au debut. Les premiers chapitres posent le sol : idee, SELECT, WHERE, ORDER, NULL, agregats, jointures. Le milieu touche ecriture et structure. Les ateliers font faire. Le quiz verifie. A chaque fin, un "A toi". Fais-le. Cinq minutes actives battent une lecture passive.

Chez DanielCraft, on forme des gens qui livrent petit, souvent, proprement - pas des collectionneurs de screenshots de dashboards jamais reconstruits. Tu ecris. Tu lances. Tu lis le resultat. Tu corriges.



Exemple : Lea lit une table clients comme une grille.

Petite histoire

Lea livrait un mini site pour un artisan. Le client demandait "qui a commande cette semaine ?". Sans SQL, elle ouvrait trois exports CSV et croisait a la main. Avec une table `commandes` et un `WHERE`, elle a repondu en deux minutes. Quarante lignes de panique en moins. Max voulait juste "voir les cinq plus gros totaux". Sam a montre `ORDER BY total DESC LIMIT 5`. L'idee est rentree sans jargon. Personne n'a dit "c'est trop". Ils ont dit "ah, c'est une question".

Erreur classique

Croire que SQL "devine" ce que tu veux. Ou vouloir tout le manuel avant d'ecrire un `SELECT nom FROM clients`. Autre piege : confondre la base (le stockage) et l'outil (l'interface ou tu tapes). Lea garde une regle : une question claire bat dix dashboards flous. DanielCraft insiste : petit, clair, testable.

⚠ ATTENTION

SQL execute exactement ce que tu ecris. Une mauvaise question donne un mauvais resultat - meme si la syntaxe est correcte.

En vrai

Imagine trois tables : `clients`, `commandes`, `produits`. Ecris sur papier une question metier en francais ("clients de Lyon", "commandes > 100", "nom du client + total"). Tu transformeras ces phrases en requetes dans les chapitres suivants.

A toi

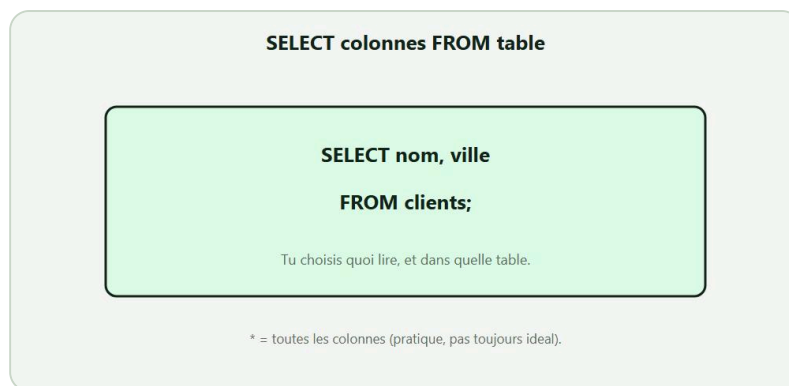
Ecris en trois phrases : (1) une question que tu aimerais poser a des donnees, (2) ce que tu acceptes d'apprendre d'abord (SELECT, WHERE...), (3) ce que tu ne feras pas encore (admin serveur, optimisation avantee). Garde ce papier pour le mini-projet. Chez DanielCraft, ce petit brief vaut plus qu'une heure de videos floues.

Exemple pour sentir

```
SELECT nom, ville  
FROM clients  
WHERE ville = 'Lyon';
```

Tu n'as pas besoin de tout comprendre maintenant. L'idée : tu choisis des colonnes, une table, un filtre. Dans ce livre, on demonte ca piece par piece, avec Lea, Max et Sam - et DanielCraft comme fil : petit, clair, testable.

Chapitre 2 - SELECT : lire des colonnes



SELECT ... FROM : choisir colonnes et table.

Le premier geste SQL utile, c'est **lire**. Pas modifier. Pas supprimer. Lire. **SELECT** choisit **quelles colonnes** tu veux voir. **FROM** dit **dans quelle table**. Point. Chez DanielCraft, on commence toujours par regarder avant d'ecrire. Lea ouvre **clients** et demande seulement **nom** et **ville**. Max a voulu ***** des le debut ; Sam lui a dit : "vois d'abord ce que tu demandes vraiment".

La forme minimale :

```
SELECT nom, ville
FROM clients;
```

Tu obtiens une grille : une ligne par client, deux colonnes. Si tu ecris **SELECT * FROM clients;**, tu prends **toutes** les colonnes. Pratique pour explorer. Moins ideal pour un rapport propre : tu ramènes du bruit, et demain une colonne sensible peut apparaitre. Prefere nommer ce dont tu as besoin.

★ A RETENIR

SELECT colonnes FROM table = choisir quoi lire, et ou. Commence par peu de colonnes.

Ce que ce n'est pas

Ce n'est pas encore un filtre (**WHERE** arrive au chapitre suivant). Ce n'est pas un tri. Ce n'est pas une jointure. Si tu ajoutes trop tot dix clauses, tu te perds. Lea coupe le superflu. Max a tendance a tout selectionner "au cas ou" ; Sam demande : "cette colonne sert-elle la question ?"

Alias et lisibilite

Tu peux renommer une colonne dans le resultat avec **AS** :

```
SELECT nom AS client, ville AS ville_client
FROM clients;
```

Utile pour un export ou un ecran. La table ne change pas. Seul l'affichage du resultat change. Tu peux aussi selectionner une expression simple plus tard ; pour l'instant, colonnes reelles suffisent.

Petite histoire

Lea devait preparer une liste "nom + ville" pour un mailing local. Elle a evite `SELECT` et a pris exactement deux colonnes. Le fichier etait leger. Max a copie un `SELECT` trouve en ligne et s'est retrouve avec dix colonnes dont un email interne. Sam a fait l'exercice a voix haute : "je choisis, je nomme, je lis". DanielCraft : le choix de colonnes est deja un acte de design.

Erreur classique

Oublier `FROM` (la base ne sait pas ou chercher). Mal orthographier le nom de table (`client` au lieu de `clients`). Mettre une virgule en trop avant `FROM`. Autre piege : croire que `SELECT` modifie la table. Non : il lit une copie logique du resultat. La table reste intacte.

⚠ ATTENTION

`SELECT` ne change pas les donnees. Si tu veux modifier, ce sera `UPDATE` / `INSERT` / `DELETE` - plus tard, avec prudence.

En vrai

Dans ton outil SQL (SQLite, PostgreSQL, interface web...), execute :

```
SELECT id, nom, ville FROM clients;
```

Puis :

```
SELECT * FROM produits;
```

Compare le confort. Note ce que `*` t'apporte et ce qu'il cache.

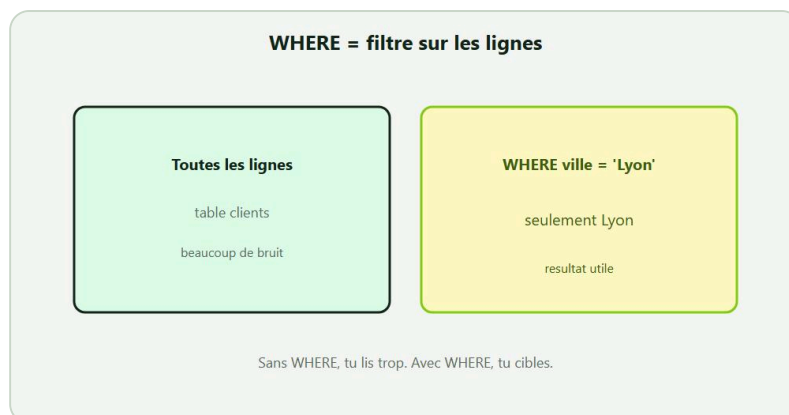
♦ ASTUCE

Quand tu explores une table inconnue, un `SELECT * ... LIMIT 10` (chapitre 4) te montre la forme sans noyer l'ecran.

A toi

Ecris trois requetes `SELECT` sur `clients` ou `produits` : (1) une seule colonne, (2) deux colonnes nommees, (3) `*` puis une version avec colonnes explicites. Dis en une phrase laquelle tu garderais pour un rapport client. Chez DanielCraft, nommer les colonnes, c'est deja clarifier l'intention.

Chapitre 3 - WHERE : filtrer les lignes



WHERE : garder seulement les lignes utiles.

Sans filtre, tu lis trop. WHERE garde **seulement les lignes** qui respectent une condition. Tu as choisi les colonnes avec SELECT. Tu cibles maintenant les lignes utiles. Chez DanielCraft, le filtre est le superpouvoir debutant. Lea filtre `ville = 'Lyon'`. Max filtre `total > 100`. Sam dit : "si ta question metier a un 'seulement', tu as besoin de WHERE".

```
SELECT nom, ville
FROM clients
WHERE ville = 'Lyon';
```

Comparaisons courantes : `=`, `<>` ou `!=`, `<`, `>`, `<=`, `>=`. Texte entre quotes simples `'Lyon'`. Nombres sans quotes : `total >= 50`.

★ A RETENIR

WHERE filtre les lignes. Sans lui, tu ramènes souvent trop de bruit.

Combiner : AND et OR

Plusieurs conditions :

```
SELECT id, client_id, total
FROM commandes
WHERE total >= 50 AND total < 200;
```

AND : les deux doivent etre vraies. OR : au moins une. Utilise des parentheses quand tu melanges :

```
SELECT nom, ville
FROM clients
WHERE (ville = 'Lyon' OR ville = 'Paris') AND nom <> '';
```

Sans parentheses, la priorite peut te surprendre. Lea les met des qu'elle doute. Max a appris apres un resultat "trop large".

IN et LIKE (leger)

```
SELECT nom, ville FROM clients WHERE ville IN ('Lyon', 'Paris', 'Nantes');
```

```
SELECT nom FROM clients WHERE nom LIKE 'A%';
```

LIKE avec **%** = "commence par / contient / finit par" selon la place du joker. Pratique. Pas magique : attention a la casse selon la base.



Exemple : Max filtre les commandes du jour.

Petite histoire

Max preparait "les commandes du jour utiles". Sans **WHERE**, il avait tout l'historique. Avec **WHERE cree_le = '2026-07-28'**, la liste tenait sur un ecran. Lea a ajoute **AND total > 0** pour ecarter les tests a zero. Sam a projete un **OR** mal parenthese : le resultat a explose. La classe a vu pourquoi la precision compte.

Erreur classique

Ecrire **WHERE ville = Lyon** sans quotes (erreur ou colonne mal interpretee). Confondre **=** et **LIKE**. Filtrer sur une colonne qui n'existe pas. Autre piege : croire que **WHERE** trie - non, c'est **ORDER BY**.

⚠ ATTENTION

Une condition fausse sur toute la table renvoie zero ligne. Ce n'est pas forcément une erreur SQL : relis ta logique metier.

En vrai

Ecris en francais : "clients de Lyon ou Paris avec un nom non vide". Traduis en **WHERE** avec parentheses. Execute. Compte les lignes. Change un **OR** en **AND** et observe.

♦ ASTUCE

Quand le resultat est etrange, simplifie : une seule condition d'abord, puis rajoute AND/OR une par une.

A toi

Sur `commandes`, ecris : (1) totaux strictement > 100 , (2) totaux entre 50 et 100 inclus, (3) `client_id` dans une petite liste `IN (...)`. Note combien de lignes chaque filtre garde. Chez DanielCraft, mesurer le filtre, c'est comprendre la question.

Chapitre 4 - ORDER BY et LIMIT



ORDER BY trie. LIMIT coupe la liste.

Tu sais lire et filtrer. Maintenant tu **ordonnes** et tu **coupes**. **ORDER BY** trie le resultat. **LIMIT** garde les N premieres lignes apres le tri (selon le moteur ; l'idee reste "top N"). Chez DanielCraft, c'est le duo "top 5" sans paniquer. Lea trie les commandes du plus gros total au plus petit. Max limite a 10 pour ne pas noyer le client. Sam rappelle : "trie d'abord, coupe ensuite - sinon tu coupes au hasard".

```
SELECT id, client_id, total
FROM commandes
ORDER BY total DESC
LIMIT 5;
```

ASC = croissant (souvent par default). **DESC** = decroissant. Tu peux trier sur plusieurs colonnes : **ORDER BY** ville ASC, nom ASC.

★ A RETENIR

ORDER BY trie. LIMIT coupe. Ensemble : top N utile.

Ou placer les clauses

Ordre mental courant :

```
SELECT ...
FROM ...
WHERE ...
ORDER BY ...
LIMIT ...;
```

Tu filtres d'abord (WHERE), tu tries le sous-ensemble, tu coupes. Si tu LIMIT sans ORDER, l'ordre peut etre instable selon le moteur. Pour un "top", ORDER est quasi obligatoire.

Petite histoire

Lea devait montrer "les 3 plus grosses commandes de Lyon". Elle a joint plus tard ; ici elle a d'abord **ORDER BY total DESC LIMIT 3** sur un extrait filtre. Max a mis LIMIT 3 sans ORDER et a cru que c'etait "les plus importantes". Sam a relance avec ORDER : la liste a change. Message reçu.

Erreur classique

Croire que LIMIT remplace WHERE. Ou trier sur une colonne absente du SELECT (parfois OK, parfois non selon le moteur - reste simple : trie sur ce que tu comprends). Autre piège : `ORDER BY 1` (numero de colonne) : court, opaque. Prefere le nom.

⚠ ATTENTION

Sans `ORDER BY`, `LIMIT` ne garantit pas un "meilleur" resultat - seulement un morceau.

En vrai

```
SELECT nom, ville FROM clients ORDER BY nom ASC LIMIT 10;
```

Puis change en `ORDER BY ville DESC, nom ASC`. Observe. Sur `commandes`, fais le top 5 des totaux.

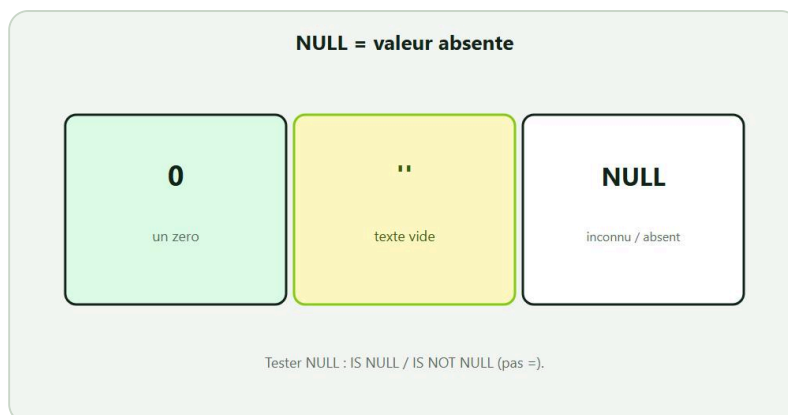
♦ ASTUCE

Pour un export "aperçu", `LIMIT 20` protege ton ecran et ta patience.

A toi

Ecris trois requetes : (1) clients tries par ville, (2) top 5 commandes, (3) top 5 apres un WHERE `total >= 50`. Explique en une phrase la difference entre (2) et (3). Chez DanielCraft, le tri sans filtre et le tri filtre ne racontent pas la meme histoire.

Chapitre 5 - NULL : la valeur absente



NULL = valeur absente. Pas zero, pas texte vide.

NULL veut dire **inconnu** ou **absent**. Ce n'est pas **0**. Ce n'est pas **" "** (texte vide). Ce n'est pas **"NULL"** en texte. Chez DanielCraft, on traite NULL comme un trou dans la grille. Lea a perdu une heure a chercher **email = NULL**. Max a cru qu'un total NULL etait zero. Sam ecrit au tableau : "NULL ne se compare pas avec =".

Pour tester :

```
SELECT nom, email
FROM clients
WHERE email IS NULL;
```

```
SELECT nom, email
FROM clients
WHERE email IS NOT NULL;
```

= NULL ne fait pas ce que tu crois (le resultat n'est pas "les lignes nulles" de facon fiable). Utilise **IS NULL** / **IS NOT NULL**.

★ A RETENIR

NULL = absent. Teste avec IS NULL / IS NOT NULL, pas avec =.

NULL dans les calculs et filtres

Une comparaison avec NULL donne souvent "inconnu", donc la ligne sort du WHERE classique. Un **total + 10** si **total** est NULL peut rester NULL. Les agregats (chapitre suivant) ont des regles : **COUNT(colonne)** ignore les NULL, **COUNT(*)** compte les lignes.

Exemple : clients sans ville renseignee :

```
SELECT id, nom
FROM clients
WHERE ville IS NULL;
```

Petite histoire

Lea exportait des clients "avec email". Son `WHERE email <> ''` laissait passer des NULL selon le moteur / les donnees. Elle a passe a `WHERE email IS NOT NULL AND email <> ''`. Max avait des commandes sans `client_id` (donnees sales) : `IS NULL` les a revelees. Sam fait l'exercice "trois boites : 0, ", NULL" a chaque promo. Personne ne confond apres.

Erreur classique

Ecrire `WHERE col = NULL`. Remplacer NULL par 0 sans reflechir (parfois OK metier, souvent mensonge). Croire que LEFT JOIN "casse" parce que des colonnes droites sont NULL - c'est normal (chapitre 8).

⚠ ATTENTION

Avant de remplacer NULL par une valeur par default, demande : "absent" et "zero" veulent-ils dire la meme chose metier ?

En vrai

Inspecte une table : cherche une colonne qui peut etre vide. Compte avec `IS NULL` et avec `= ''` si c'est du texte. Compare les comptes.

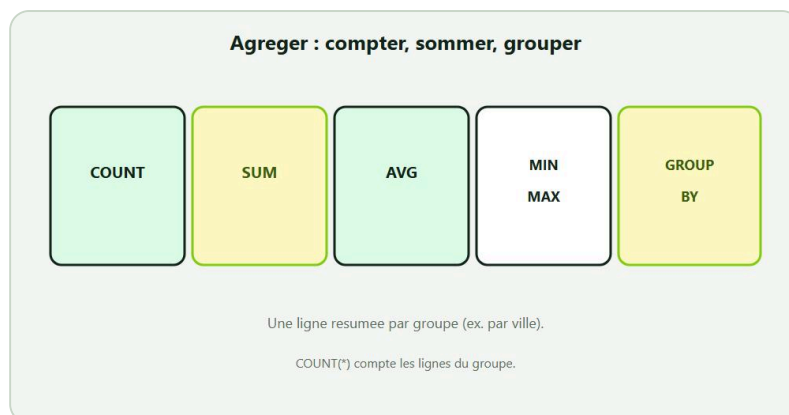
♦ ASTUCE

Quand un JOIN "perd" des lignes ou en "invente" des NULL, dessine la grille : NULL a droite = pas de match, pas forcement bug.

A toi

Ecris : (1) lignes ou une colonne est NULL, (2) lignes ou elle ne l'est pas, (3) une phrase metier expliquant ce que NULL signifie chez toi (email manquant, commande non livree...). Chez DanielCraft, nommer le trou evite de le remplir n'importe comment.

Chapitre 6 - Agregats et GROUP BY



COUNT SUM AVG MIN MAX + GROUP BY.

Parfois tu ne veux pas chaque ligne : tu veux un **resume**. Combien de clients ? Somme des totaux ? Moyenne ? Minimum / maximum ? Les fonctions d'agregat repondent. Chez DanielCraft, c'est le passage "liste" vers "chiffre utile". Lea compte les commandes par client. Max somme les totaux du mois. Sam dit : "agreger, c'est accepter de perdre le detail pour gagner la vue".

```
SELECT COUNT(*) AS nb
FROM clients;
```

```
SELECT SUM(total) AS ca, AVG(total) AS moyenne, MIN(total) AS mini, MAX(total) AS maxi
FROM commandes;
```

COUNT(*) compte les lignes. COUNT(email) compte les emails non NULL. Nuance utile apres le chapitre NULL.

★ A RETENIR

COUNT SUM AVG MIN MAX resument. GROUP BY decoupe le resume par groupe.

GROUP BY

Pour un chiffre **par ville** ou **par client** :

```
SELECT ville, COUNT(*) AS nb_clients
FROM clients
GROUP BY ville
ORDER BY nb_clients DESC;
```

```
SELECT client_id, SUM(total) AS ca
FROM commandes
GROUP BY client_id
ORDER BY ca DESC;
```

Regle d'or debutant : dans le `SELECT`, chaque colonne non agregee doit etre dans le `GROUP BY` (selon le mode SQL strict). Ne melange pas `nom` libre et `SUM(total)` sans grouper correctement.

HAVING (aperçu)

`WHERE` filtre **avant** l'agregat. `HAVING` filtre **apres**, sur le resultat groupe :

```
SELECT client_id, SUM(total) AS ca
FROM commandes
GROUP BY client_id
HAVING SUM(total) >= 200;
```

Tu n'as pas besoin de tout maitriser maintenant. Retiens : `WHERE` sur lignes brutes, `HAVING` sur groupes.

Petite histoire

Lea devait repondre "combien de clients par ville ?". Sans `GROUP BY`, elle comptait a la main. Avec, trois secondes. Max a mis `SELECT nom, SUM(total)` sans `GROUP BY` et a pris une erreur (ou un resultat incoherent selon le moteur). Sam a corrige au tableau. Message : le groupe doit etre explicite.

Erreur classique

Oublier `GROUP BY` alors que tu agreges avec une colonne libre. Utiliser `WHERE` a la place de `HAVING` pour filtrer sur `SUM(...)`. Croire que `COUNT(*)` et `COUNT(col)` sont toujours egaux.

⚠ ATTENTION

Agreger sans definir le groupe, c'est melanger des pommes et des paniers. Ecris d'abord "par quoi je regroupe ?".

En vrai

Compte les commandes. Somme les totaux. Puis somme par `client_id`. Compare une ligne globale et plusieurs lignes groupees.

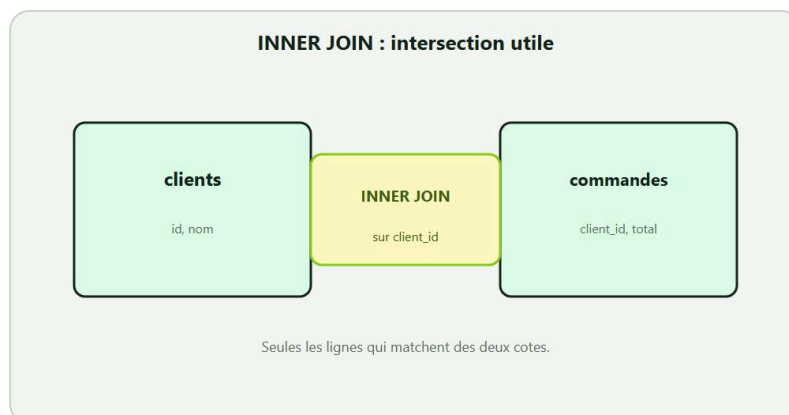
💡 ASTUCE

Pour verifier un `GROUP BY`, relance sans agregat avec `WHERE` sur un groupe (un `client_id`) et compare a la main.

A toi

Ecris : (1) `COUNT(*)` sur `produits`, (2) `SUM(total)` global sur `commandes`, (3) `SUM(total)` par `client_id` avec `ORDER BY`. Note le client en tete. Chez DanielCraft, un chiffre verifie bat un dashboard joli non compris.

Chapitre 7 - INNER JOIN : croiser deux tables



INNER JOIN : croiser deux tables sur une cle.

Une table seule ne raconte pas tout. `clients` a les noms. `commandes` a les totaux et un `client_id`. Pour afficher **nom + total**, tu **joins** sur la cle commune. `INNER JOIN` garde seulement les lignes ou le match existe des deux cotes. Chez DanielCraft, c'est le geste "relier sans magie". Lea joint pour facturer. Max voyait des ids ; Sam lui a montre le nom via JOIN. L'oeil a change.

```
SELECT clients.nom, commandes.total, commandes.cree_le
FROM clients
INNER JOIN commandes ON clients.id = commandes.client_id;
```

`ON` dit comment ca matche. Ici : l'id client egal a la cle etrangere de la commande.

★ A RETENIR

INNER JOIN = intersection utile. Seulement les lignes qui matchent des deux cotes.

Alias de tables

Pour alleger :

```
SELECT c.nom, co.total
FROM clients AS c
INNER JOIN commandes AS co ON c.id = co.client_id
WHERE co.total >= 50
ORDER BY co.total DESC;
```

Tu prefixes les colonnes (`c.nom`) pour eviter l'ambiguite si les deux tables ont `id`.

Pourquoi INNER "perd" des lignes

Un client sans commande n'apparait pas en INNER JOIN. Une commande orpheline (`client_id` inconnu) non plus. Ce n'est pas forcément un bug : c'est la definition. Si tu veux garder les clients sans commande, tu passeras au LEFT JOIN (chapitre suivant).



Exemple : Sam relie clients et commandes.

Petite histoire

Sam a dessiné deux cercles au tableau : clients et commandes. L'intersection = INNER. Lea a livré un export "qui a commande" sans les curieux inactifs. Max a oublié le **ON** et a créé un produit cartésien monstrueux (chaque client avec chaque commande). La base a ralenti. Leçon : jamais de JOIN sans condition claire.

⚠ ATTENTION

Un JOIN sans bon **ON** (ou avec une mauvaise clé) multiplie les lignes. Vérifie avec COUNT avant/après.

Erreur classique

Joindre sur les mauvais champs (`clients.nom = commandes.total`). Oublier le préfixe et avoir `id` ambigu. Filtrer après coup sans comprendre pourquoi des clients manquent (INNER).

En vrai

Compte `SELECT COUNT(*) FROM commandes` . Puis compte le INNER JOIN clients/commandes. Si les nombres divergent fort, inspecte les `client_id` orphelins.

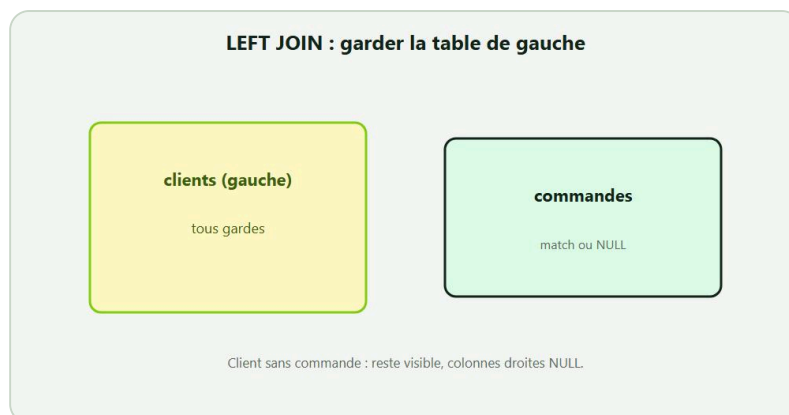
♦ ASTUCE

Écris d'abord la phrase : "chaque commande appartient à un client via `client_id`". Puis traduis en ON.

A toi

Écris un INNER JOIN `clients` + `commandes` affichant nom, total, date. Ajoute WHERE total >= 50. Limite à 10. Chez DanielCraft, une jointure lue à voix haute ("sur `client_id`") évite le flou.

Chapitre 8 - LEFT JOIN : garder la gauche



LEFT JOIN : garder la gauche meme sans match.

INNER JOIN ne garde que les matchs. **LEFT JOIN** garde **toutes** les lignes de la table de **gauche**, meme sans match a droite. Les colonnes de droite deviennent **NULL** si pas de commande. Chez DanielCraft, c'est la requete "tous les clients, meme ceux qui n'ont rien commande". Lea l'utilise pour relancer les inactifs. Max cherchait un bug ; c'etait juste un LEFT avec des NULL. Sam dit : "gauche = priorite de visibilite".

```
SELECT c.nom, co.id AS commande_id, co.total
FROM clients AS c
LEFT JOIN commandes AS co ON c.id = co.client_id
ORDER BY c.nom;
```

Un client sans commande : une ligne avec **commande_id** et **total** a NULL.

★ A RETENIR

LEFT JOIN = tous a gauche. Droite match ou NULL.

Trouver ceux sans match

Pattern classique :

```
SELECT c.nom
FROM clients AS c
LEFT JOIN commandes AS co ON c.id = co.client_id
WHERE co.id IS NULL;
```

Tu gardes la gauche, tu filtres ou la droite est absente. Ideal pour "clients sans commande".

INNER vs LEFT : choisir

- Tu veux seulement les couples existants -> INNER.
- Tu veux inventorier la gauche complete -> LEFT.

Ne choisis pas LEFT "par peur" : trop de NULL brouille le rapport. Choisis selon la question metier.

Petite histoire

Lea a envoyé un mailing à "tous les clients" en INNER JOIN par erreur : les nouveaux sans commande étaient invisibles. Passage en LEFT : la liste était complète. Max a paniqué en voyant des NULL ; Sam a sourit : "c'est le signal 'pas de commande', pas une corruption". DanielCraft : NULL ici est une information.

Erreur classique

Mettre la table "importante" à droite et s'étonner qu'elle disparaisse (ce serait un autre type de jointure). Filtrer `WHERE co.total > 0` après LEFT sans faire attention : tu retransformes souvent le LEFT en INNER (les NULL sortent). Pour filtrer la droite en gardant les sans-match, il faut parfois mettre la condition dans le `ON`, ou accepter le comportement.

⚠ ATTENTION

Un WHERE sur une colonne de droite peut annuler l'intérêt du LEFT JOIN. Relis le résultat.

En vrai

Compare le nombre de lignes : INNER vs LEFT sur clients/commandes. Liste les clients avec `WHERE co.id IS NULL`.

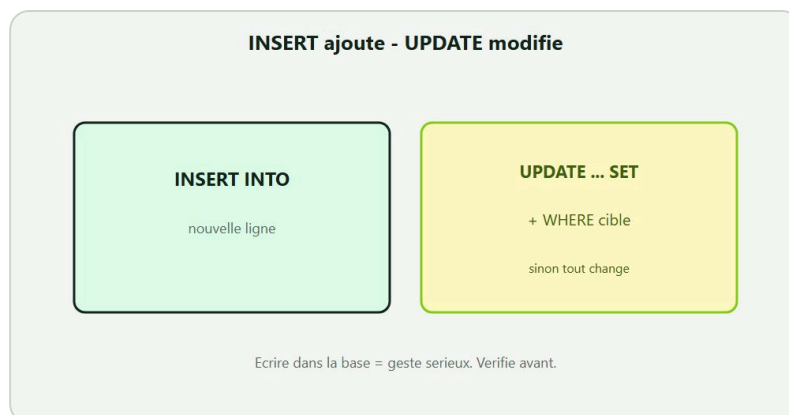
💡 ASTUCE

Dis à voix haute : "je veux tous les clients". Si oui, clients à gauche en LEFT JOIN.

A toi

Ecris (1) LEFT JOIN nom + total, (2) version "sans commande" avec IS NULL, (3) une phrase : quand tu choisirais INNER à la place. Chez DanielCraft, le choix de jointure, c'est le choix de la question.

Chapitre 9 - INSERT et UPDATE



INSERT ajoute. UPDATE modifie (avec WHERE).

Jusqu'ici tu lisais. Maintenant tu **ecris** dans la base. **INSERT** ajoute une ligne. **UPDATE** modifie des lignes existantes. Chez DanielCraft, écrire est un geste sérieux : tu changes la réalité partagée. Lea double-check avant. Max a mis à jour sans WHERE une fois - histoire courte, leçon longue. Sam impose : "SELECT d'abord, écriture ensuite".

```
INSERT INTO clients (nom, ville)
VALUES ('Ines', 'Nantes');
```

```
UPDATE clients
SET ville = 'Lyon'
WHERE id = 3;
```

★ A RETENIR

INSERT ajoute. UPDATE modifie. Toujours cibler avec WHERE pour UPDATE.

UPDATE : le WHERE obligatoire mentalement

```
-- Dangereux : touche potentiellement toute la table
-- UPDATE clients SET ville = 'Lyon';

-- Sain : une cible claire
UPDATE clients
SET ville = 'Lyon'
WHERE id = 3;
```

Reflexe : écris d'abord le SELECT équivalent (`SELECT * FROM clients WHERE id = 3`), vérifie, puis remplace SELECT par UPDATE ... SET.

Plusieurs colonnes

```
UPDATE commandes
SET total = 120, cree_le = '2026-07-28'
WHERE id = 42;
```

```
INSERT INTO produits (nom, prix)
VALUES ('Joint silicone', 4.50);
```

Les colonnes et les valeurs doivent s'aligner (nombre et ordre). Respecte les types (chapitre 12) et les clés (chapitre 11).

Petite histoire

Max voulait corriger une ville. Il a oublié WHERE. Tous les clients sont devenus "Lyon". Sauvegarde ou transaction selon l'outil l'a sauvé ; la sueur aussi. Lea insère toujours une ligne de test puis SELECT pour vérifier. Sam chronomètre le "SELECT miroir" avant tout UPDATE en atelier. DanielCraft : la lenteur de vérification bat la vitesse de catastrophe.

Erreur classique

UPDATE sans WHERE. INSERT avec mauvais nombre de valeurs. Mettre un texte dans une colonne nombre. Croire que INSERT "remplace" une ligne existante (non : ça ajoute ; le remplacement propre passe souvent par UPDATE ou règles spécifiques).

⚠ ATTENTION

Sur une base partagée, un UPDATE large peut casser le travail de tout le monde. Cible étroitement.

En vrai

Dans un environnement de test seulement : INSERT un client fictif, SELECT-le, UPDATE sa ville, SELECT encore. Ne joue pas ça sur la prod.

✦ ASTUCE

Garde une transaction / un backup mental : "puis-je annuler ?" Si non, n'exécute pas encore.

A toi

Écris sur papier (sans exécuter en prod) : (1) un INSERT clients, (2) un UPDATE avec WHERE id, (3) le SELECT de vérification avant UPDATE. Chez DanielCraft, le papier avant le clic sauve des soirées.

Chapitre 10 - DELETE : retirer des lignes

DELETE : retirer des lignes

DELETE FROM commandes

WHERE id = 42;

Sans WHERE : risque de tout effacer.

Reflexe : SELECT d'abord, DELETE ensuite.

DELETE retire des lignes. Toujours verifier WHERE.

DELETE enleve des **lignes**. Pas la table entiere (ca serait **DROP**, hors focus ici). Chez DanielCraft, DELETE est le chapitre du frein a main. Lea n'efface qu'apres SELECT. Max a failli vider **commandes**. Sam ecrit en gros : WHERE.

```
DELETE FROM commandes
WHERE id = 42;
```

★ A RETENIR

DELETE retire des lignes. SELECT d'abord. WHERE presque toujours.

Le reflexe SELECT miroir

```
SELECT * FROM commandes WHERE id = 42;
-- si la ligne est la bonne :
DELETE FROM commandes WHERE id = 42;
```

Pour un lot :

```
SELECT * FROM commandes WHERE total = 0 AND cree_le < '2025-01-01';
-- puis seulement si le compte est attendu :
DELETE FROM commandes WHERE total = 0 AND cree_le < '2025-01-01';
```

Compte les lignes du SELECT. Si tu en vois 8000 et tu en attendais 8, tu stops.

DELETE sans WHERE

```
-- Tres dangereux
-- DELETE FROM commandes;
```

Selon le moteur / droits, tu peux vider la table. Ne le fais pas "pour tester" hors bac a sable.

Petite histoire

Lea devait retirer des commandes de test a total 0. Elle a SELECT, vu 12 lignes, DELETE avec le meme WHERE, re-SELECT, zero reste. Max a lance un DELETE avec une faute de frappe dans le WHERE qui matchait trop large. Heureusement, environnement de stage. Sam raconte cette histoire a chaque promo. Elle marche.

Erreur classique

DELETE sans WHERE. Confondre DELETE (lignes) et DROP (objet). Effacer avant de comprendre les cles etrangeres (une commande peut etre liee ; la base peut refuser ou cascade selon schema).

⚠ ATTENTION

Une fois commit / hors transaction, DELETE peut etre definitif. Verifie deux fois.

En vrai

En test : inserer une ligne jetable, la SELECT, la DELETE, SELECT encore (zero ligne). Note le rythme.

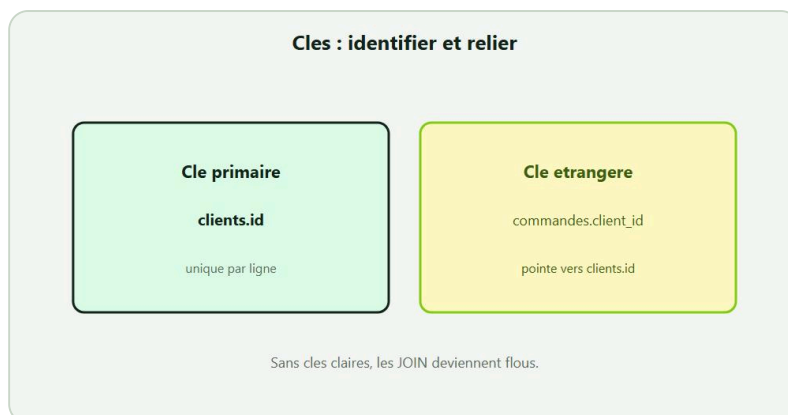
♦ ASTUCE

Si tu hesites, exporte d'abord les lignes visees (`SELECT ...`). Tu pourras re-INSERT si besoin.

A toi

Ecris le duo SELECT + DELETE pour supprimer une commande d'id connu. Ajoute une phrase "je n'execute en vrai que si...". Chez DanielCraft, la phrase frein fait partie de la competence.

Chapitre 11 - Cles primaires et etrangeres



Cle primaire et cle etrangere : le lien.

Une **cle primaire** identifie une ligne de facon unique (souvent `id`). Une **cle etrangere** pointe vers la cle d'une autre table. Chez DanielCraft, on les lit comme des etiquettes et des fils : sans etiquette claire, les jointures deviennent du bricolage.

Exemple invente : table `clients` avec `id` unique. Table `commandes` avec `client_id` qui pointe vers `clients.id`. Max joint ainsi. Sam oublie la cle et croise n'importe comment : resultats absurdes.

Ce que ce n'est pas

Ce n'est pas "un id magique qui reparte tout". Ce n'est pas obligatoire de tout comprendre en DBA des le jour 1. C'est le contrat minimal pour croiser sans mentir.

Petite histoire

Lea cree `produits(id, nom)`. Puis `lignes_commande(commande_id, produit_id, qte)`. Elle explique a Max : "la cle dit qui est qui". Max arrete de joindre sur le prenom.

Erreur classique

Joindre sur une colonne non unique (prenom, ville). Ou croire que deux tables "se comprennent" sans cle.

En vrai

Dessine deux boites `clients` et `commandes`. Fleche de `client_id` vers `id`.

A toi

Ecris la cle primaire et la cle etrangere de ton mini schema perso.

★ A RETENIR

Cle primaire = identite. Cle etrangere = lien vers une autre table.

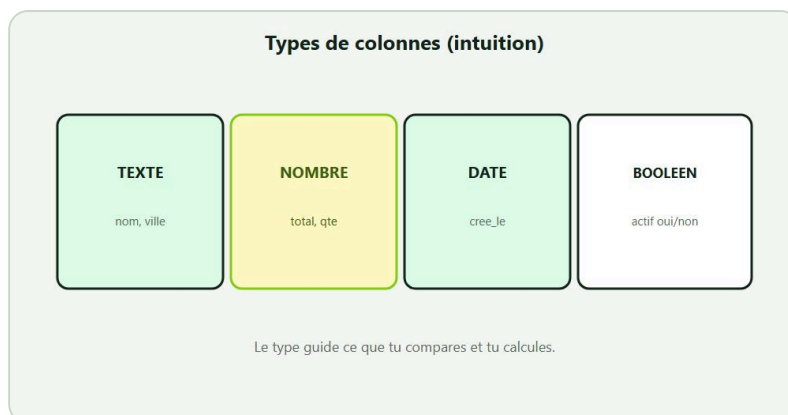
⚠ ATTENTION

Joindre sans cle claire = resultats qui mentent gentiment.

♦ **ASTUCE**

Nomme `quelquechose_id` pour rendre le lien visible.

Chapitre 12 - Types de colonnes



Types de colonnes : texte, nombre, date...

Une colonne a un **type** : texte, nombre entier, decimal, date, booleen... Le type dit ce que tu peux comparer et calculer. Chez DanielCraft, on ne fait pas un catalogue exhaustif : on lit l'intention. `prix` en texte, c'est un piège. `prix` en nombre, tu peux sommer.

Lea stocke `ville` en texte. Max met `quantite` en entier. Sam range une date de commande en type date (ou texte ISO si l'outil est limite) et evite "hier" en clair dans la colonne.

Ce que ce n'est pas

Ce n'est pas "tous les dialectes SQL sont identiques". MySQL, Postgres, SQLite ont des nuances. L'idée reste : choisis un type qui match l'usage.

Petite histoire

Max compare `prix > '20'` alors que `prix` est texte. L'ordre lexicographique le trompe. Il passe en nombre. Lea sourit : le type a parlé.

Erreur classique

Mettre des montants en texte. Ou des dates en format francais meuble impossible a trier.

En vrai

Liste trois colonnes de `clients` et propose un type pour chacune.

A toi

Corrige une mauvaise idee : "telephone en nombre". Ecris pourquoi texte est souvent mieux.

★ A RETENIR

Le type de colonne guide les comparaisons et les calculs.

⚠ ATTENTION

Montant en texte = tris et sommes foireux.

♦ **ASTUCE**

Pour debuter : texte, entier, decimal, date. Le reste viendra.

Chapitre 13 - Mini-projet : clients et commandes



Mini-projet : requetes sur clients et commandes.

Objectif : un petit jeu de tables inventees et des requetes qui repondent a de vraies questions. Chez DanielCraft, le mini-projet n'est pas un ERP. C'est : lire, filtrer, agreger, joindre.

Schema invente

`clients(id, prenom, ville)` `commandes(id, client_id, montant, date_cmd)`

Donnees inventees : 5 clients, 8 commandes. Tu peux les ecrire sur papier si tu n'as pas encore de base.

Questions a repondre

1. 1. Liste des clients de Paris.
2. 2. Top 3 des commandes les plus cheres.
3. 3. Total des montants par ville (JOIN + GROUP BY).
4. 4. Clients sans commande (LEFT JOIN + filtre NULL) - bonus.

Lea fait 1 et 2. Max bloque sur 3 puis croise. Sam verifie chaque resultat a la main sur 3 lignes.



Exemple : Lea, Max et Sam livrent le rapport.

Petite histoire

Nora (du livre finance) demanderait le chiffre. Ici Max livre : "Paris = 420 EUR ce mois (donnees inventees)". Le SQL a servi le discours, pas l'inverse.

Erreur classique

Ecrire la requete finale sans tester `SELECT * LIMIT 5` avant. Ou oublier le lien `client_id`.

En vrai

Ecris les quatre requetes sur papier, meme imparfaites.

A toi

Une page "Mini-projet SQL - date" avec schema + 4 reponses.

★ A RETENIR

Mini-projet = questions metier + requetes + verification manuelle.

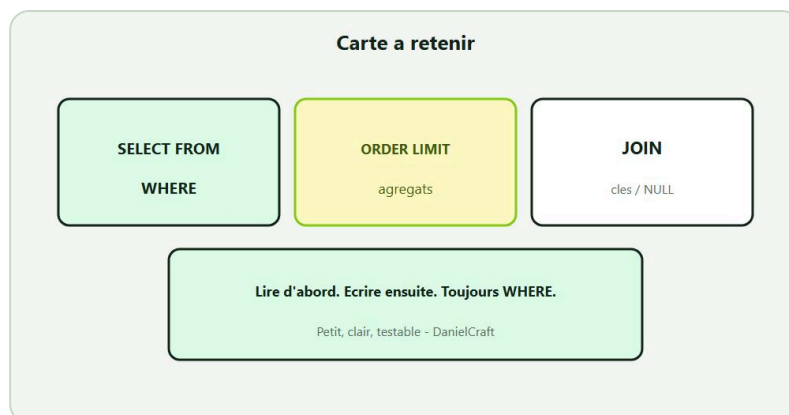
⚠ ATTENTION

Sans verifier 2-3 lignes a la main, tu peux "reussir" avec un JOIN faux.

♦ ASTUCE

Commence toujours par SELECT et LIMIT avant d'agreger.

Chapitre 14 - Retenir l'essentiel



SELECT WHERE JOIN : la carte a garder.

SQL, c'est poser une question claire a des tables. Tu choisis des colonnes (**SELECT**), une table (**FROM**), tu filtres (**WHERE**), tu tries (**ORDER BY**), tu limites (**LIMIT**). Tu geres l'absence (**NULL**). Tu resumes (**COUNT** , **SUM** , **GROUP BY**). Tu croises (**JOIN**). Tu modifies avec prudence (**INSERT** , **UPDATE** , **DELETE** + **WHERE**). Chez DanielCraft : petit, clair, verifie.

Lea retient : table = grille. Max retient : WHERE avant DELETE. Sam retient : JOIN sur une cle.

Carte courte

- SELECT / FROM / WHERE / ORDER / LIMIT
- NULL n'est pas 0
- GROUP BY avec agregats
- INNER vs LEFT JOIN
- Cles primaire / etrangere
- Toujours verifier un echantillon

A toi

Recopie ta phrase perso en une ligne.

★ A RETENIR

Question claire + cle claire + echantillon verifie.

◆ ASTUCE

Relis WHERE et JOIN avant toute requete de prod inventee.

Chapitre 15 - Atelier : SELECT



Atelier : lire des colonnes sans paniquer.

Atelier lecture seule. Table inventee `produits(id, nom, prix, stock)`.

Etapes

1. 1. Ecris `SELECT * FROM produits;` (esprit)
2. 2. Selectionne seulement `nom, prix`
3. 3. Alias : `prix AS prix_eur`
4. 4. Verifie mentalement 3 lignes

Chez DanielCraft, l'atelier SELECT apprend a ne pas tout afficher "parce que c'est plus simple". Lea choisit les colonnes utiles. Max commence toujours par `*` puis resserre.

A toi

Ecris deux SELECT differents sur `produits`.

★ A RETENIR

SELECT choisit les colonnes. Moins = plus lisible.

⚠ ATTENTION

`SELECT *` en exploratoire OK. En rapport final, precis mieux.

♦ ASTUCE

Nomme les alias pour le lecteur humain.

Chapitre 16 - Atelier : filtrer avec WHERE



Atelier : filtrer avec WHERE.

Table inventee `commandes(id, client_id, montant, ville_livraison)`.

Missions

1. Montants > 50
2. Ville = 'Lyon'
3. Combine : Lyon ET montant >= 30
4. Liste triee par montant desc, 5 lignes

Max ecrit d'abord le filtre en francais ("commandes de Lyon pas trop petites") puis le traduit en SQL. Sam teste chaque condition separement.

A toi

Ecris les quatre requetes. Entoure le WHERE d'un trait sur papier.

★ A RETENIR

WHERE garde les lignes utiles. Une condition claire bat trois floues.

⚠ ATTENTION

Oublier les guillemets sur un texte = erreur ou surprise.

♦ ASTUCE

Traduis la phrase metier avant le SQL.

Chapitre 17 - Atelier : jointure



Atelier : croiser deux tables.

Tables : `clients(id, prenom)`, `commandes(id, client_id, montant)`.

Missions

1. 1. INNER JOIN : prenom + montant
2. 2. Total par prenom (JOIN + GROUP BY)
3. 3. LEFT JOIN : clients meme sans commande
4. 4. Compte combien de clients sans commande

Sam dessine la fleche `client_id -> id` avant d'ecrire. Lea verifie avec 2 clients connus.

A toi

Ecris 1 et 3. Si 2 bloque, relis agregats.

★ A RETENIR

JOIN = croiser sur une cle. Dessine avant d'ecrire.

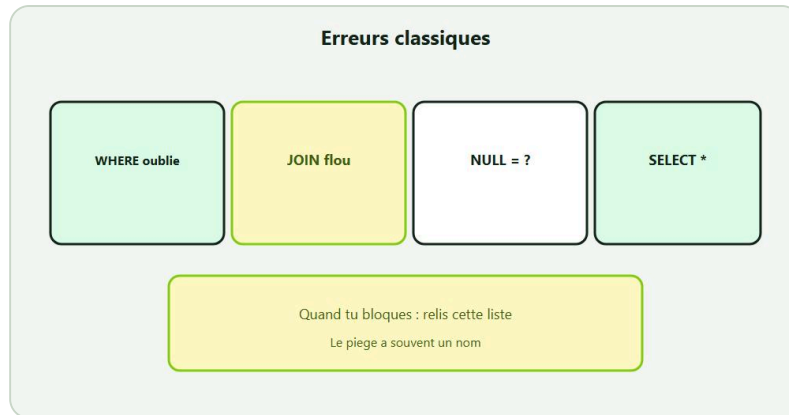
⚠ ATTENTION

JOIN sans condition (ou mauvaise cle) = explosion de lignes.

♦ ASTUCE

Prefixe les colonnes : `clients.prenom`, `commandes.montant`.

Chapitre 18 - Erreurs classiques



Pièges : WHERE oublié, JOIN flou, NULL.

Oublier `WHERE` sur un `UPDATE` / `DELETE`. Comparer avec `=` à `NULL` au lieu de `IS NULL`. Joindre sur le mauvais champ. Faire confiance à un total sans regarder 3 lignes. Mettre des prix en texte. Chez DanielCraft, ces pièges reviennent plus souvent que les "gros bugs savants".

Lea a failli vider une table de test sans WHERE. Max a joint sur `prenom`. Sam a somme des NULL sans comprendre.

Petite histoire

Un ami de Max montre un dashboard "CA du mois". Lea demande la requête. Il y a un JOIN en trop. Le chiffre était beau. Faux.

A toi

Coche les pièges que tu as déjà froles (même sur papier).

★ A RETENIR

Les erreurs SQL classiques sont humaines : WHERE, NULL, JOIN, types.

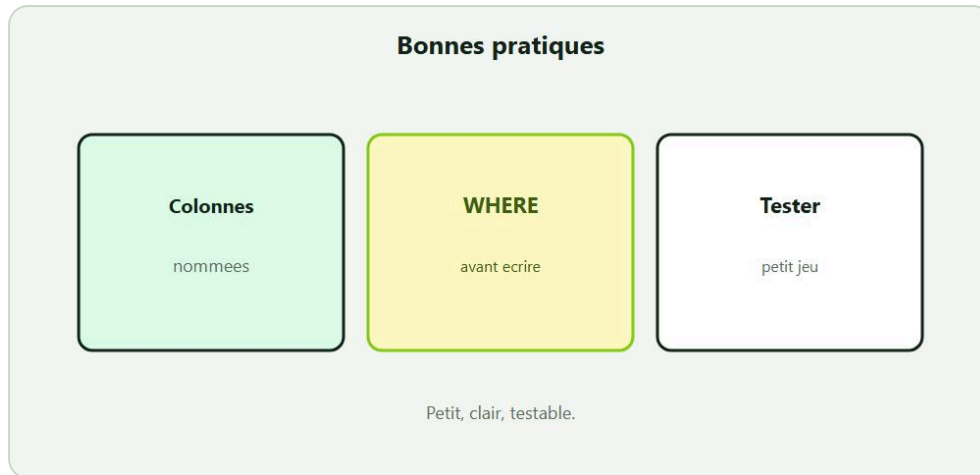
⚠ ATTENTION

Un beau chiffre ne prouve pas une bonne requête.

♦ ASTUCE

Garde une checklist : WHERE? cle? échantillon? type?

Chapitre 19 - Bonnes pratiques



Pratiques : petit, clair, testable.

1. Ecrire la question en francais d'abord.
2. Commencer petit : SELECT + LIMIT.
3. Filtrer tot avec WHERE.
4. Joindre sur des cles, pas sur du flou.
5. Nommer clairement tables et alias.
6. Verifier un echantillon a la main.
7. Se mefier de NULL dans les agregats.
8. Prefixer les colonnes en JOIN.
9. Ne jamais UPDATE/DELETE sans WHERE verifie.
10. Documenter la requete en une phrase au-dessus.

Lea affiche la checklist. Max la suit le jour du stress. Sam l'enseigne a ses eleves.

A toi

Choisis trois pratiques prioritaires. Papier a cote du clavier.

★ A RETENIR

Petit, clair, verifie.

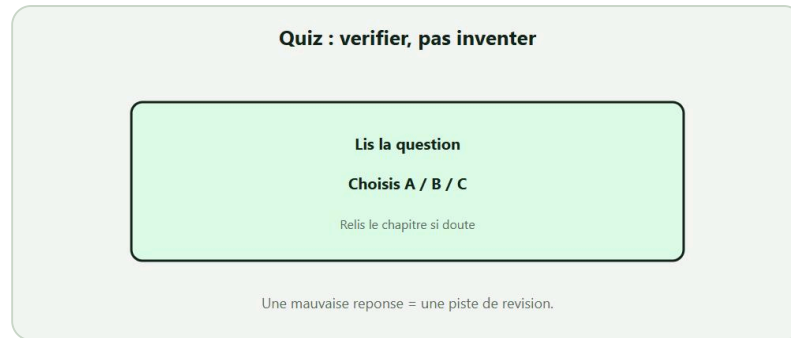
⚠ ATTENTION

La pratique se juge le jour du chiffre urgent.

♦ ASTUCE

Une phrase metier au-dessus de chaque requete importante.

Quiz final



Quiz : verifier le filtre SQL.

Douze questions. Chez DanielCraft, un sans-faute n'autorise pas a ecraser une table. Ca confirme le filtre.

Question 1

SQL sert surtout a :

- A) Dessiner des pages web
- B) Interroger et manipuler des donnees en tables
- C) Remplacer HTML

Question 2

SELECT choisit :

- A) Les lignes uniquement
- B) Les colonnes (et la forme du resultat)
- C) Le mot de passe

Question 3

WHERE sert a :

- A) Trier
- B) Filtrer les lignes
- C) Creer une table

Question 4

NULL veut dire :

- A) Zero
- B) Valeur absente
- C) Chaine vide toujours

Question 5

COUNT(*) compte :

- A) Les colonnes

- B) Les lignes du groupe / resultat
- C) Les index seulement

Question 6

INNER JOIN garde :

- A) Toutes les lignes des deux tables toujours
- B) Les lignes qui matchent des deux cotes
- C) Seulement la table de droite

Question 7

LEFT JOIN garde :

- A) Toute la table de gauche, meme sans match
- B) Seulement les matches
- C) Rien

Question 8

Avant un DELETE, tu dois surtout :

- A) Fermer les yeux
- B) Verifier le WHERE (et tester en SELECT)
- C) Ajouter ORDER BY

Question 9

Une cle primaire :

- A) Identifie une ligne de facon unique
- B) Est toujours un prenom
- C) Remplace WHERE

Question 10

Joindre sur le prenom :

- A) Est toujours sur
- B) Est risque si non unique
- C) Est obligatoire

Question 11

LIMIT sert a :

- A) Filtrer comme WHERE
- B) Couper le nombre de lignes renvoyees
- C) Supprimer une table

Question 12

Bonne pratique :

- A) UPDATE sans WHERE
- B) Verifier un echantillon a la main
- C) SELECT * partout en prod sans reflechir

Corrige

1B, 2B, 3B, 4B, 5B, 6B, 7A, 8B, 9A, 10B, 11B, 12B.

★ A RETENIR

SELECT, WHERE, JOIN, NULL, prudence sur DELETE.

⚠ ATTENTION

Sans-faute != permission d'ecraser des donnees.

♦ ASTUCE

Refais 4, 7, 8.

Bravo



Bravo. Tu interroges des tables sans magie.

Tu as termine **SQL - Les bases**.

Tu sais lire une table, filtrer, trier, agreger, joindre, et toucher INSERT/UPDATE/DELETE avec prudence. Ce n'est pas un diplome DBA. C'est un filtre a betises sur les donnees. Chez DanielCraft, on prefere une requete courte verifiee a un monstre opaque.

Suite

Niveau **Intermediaire** (sous-requetes, fenetres, perf) plus tard dans le pack Data/SQL. Ou enchaîne Python pratique pour croiser CSV et requetes. Ou reste a relire JOIN et WHERE : deja puissant.

A toi

Un geste : refaire le mini-projet sur papier, ou ecrire ta checklist DELETE.

★ A RETENIR

Question claire. Cle claire. Echantillon verifie.

♦ ASTUCE

Relis jointures le jour ou un chiffre "parait trop beau".

Tu as les briques. Maintenant, interroge.